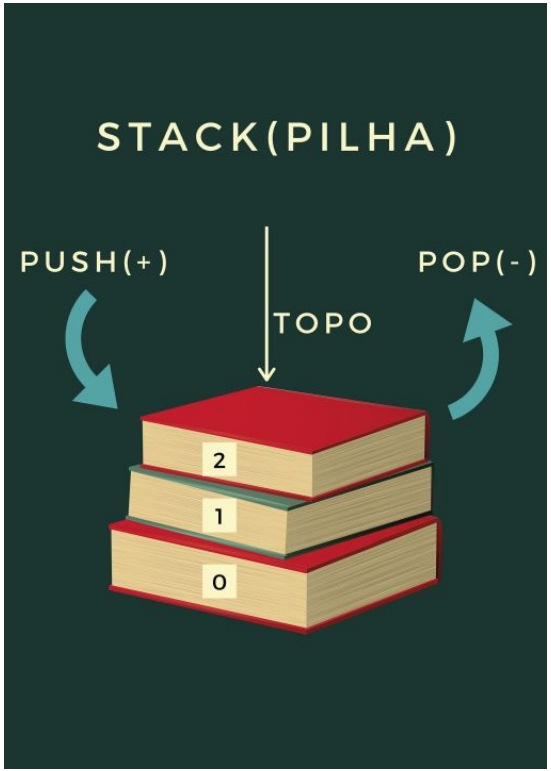


ED1 04: Pilhas

Fabio Irigon Pereira

Pilhas



Conceito de Pilha

Uma pilha (stack) é uma estrutura de dados LIFO (Last In, First Out), onde o último elemento inserido é o primeiro a ser removido.

- Inserção (push) e remoção (pop) ocorrem apenas no topo da pilha.
- Possui operações auxiliares como peek/top (ver o elemento no topo sem removê-lo).
- Pode ser implementada usando arrays (estática) ou listas ligadas (dinâmica).

Casos de uso:

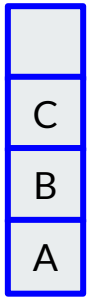
Histórico do Navegador: O botão "Voltar".

"Undo": Ctrl+Z em editores de texto.

Chamadas de Função (Call Stack).

Pilhas: Estática Vs Dinâmica

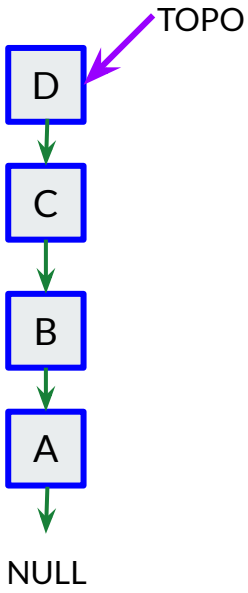
Pilha Estática
(array)



TOPO: 2
ou
QTD: 3

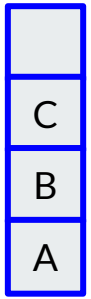


Pilha Dinâmica
(Lista Ligada)



Pilhas: Estática Vs Dinâmica

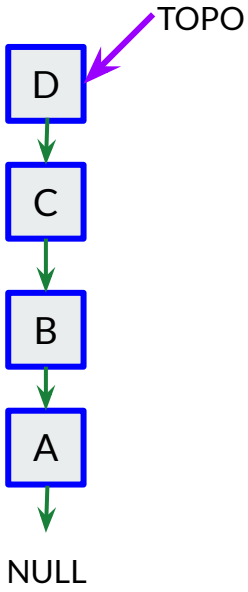
Pilha Estática
(array)



TOPO: 2
ou
QTD: 3



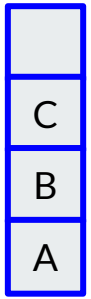
Pilha Dinâmica
(Lista Ligada)



Nota: se a pilha estiver vazia,
qual o valor do topo?

Pilhas: Estática Vs Dinâmica

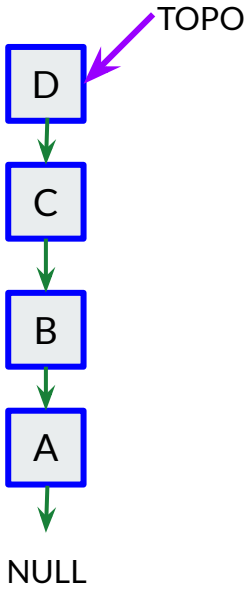
Pilha Estática
(array)



TOPO: 2
ou
QTD: 3



Pilha Dinâmica
(Lista Ligada)



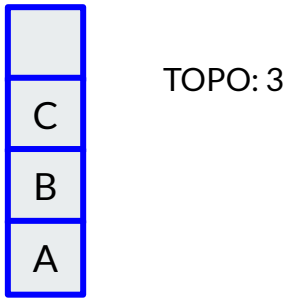
Nota: se a pilha estiver vazia,
qual o valor do topo?
R: -1

Operações Fundamentais

- Push (Inserção)
  - Adiciona um elemento no topo da pilha.
- Pop (Remoção)
  - Remove e retorna o elemento do topo da pilha.
- Peek/Top
  - Retorna o elemento do topo sem removê-lo.
- isEmpty
  - Verifica se a pilha está vazia.

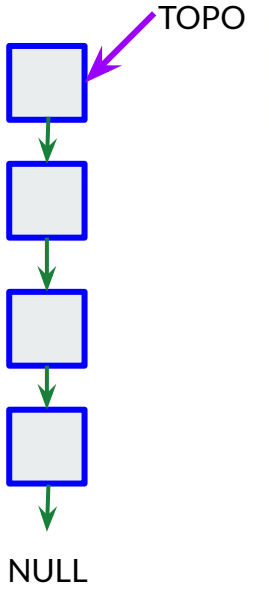
Elemento da pilha

```c
// Estática
typedef struct {
   char nome[50];
   int idade;
} Pessoa;
```



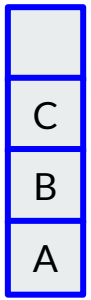
TOPO: 3

```c
// Dinâmica
typedef struct No{
   char nome[50];
   int idade;
   struct No* prox; //Ponteiro para o próximo
} No;
```



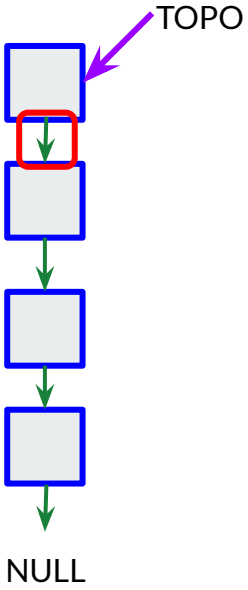
Elemento da pilha

```c
// Estática
typedef struct {
    char nome[50];
    int idade;
} Pessoa;
```



TOPO: 3

```c
// Dinâmica
typedef struct No{
    char nome[50];
    int idade;
    struct No* prox; //Ponteiro para o próximo
} No;
```



Operações em pilhas estáticas

fonte: https://github.com/arbackes/Estrutura-de-Dados-em-C

Definição do elemento e funções (estática).

```
#define MAX 100
typedef struct aluno{
   int idade;
   char nome[30];
} aluno;
```

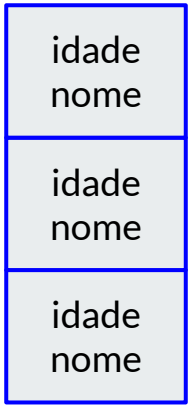


Definição do elemento e funções (estática).

```
#define MAX 100
typedef struct aluno{
   int idade;
   char nome[30];
} aluno;

typedef struct Pilha{
   int qtd;
   struct aluno dados[MAX];
} Pilha;
```



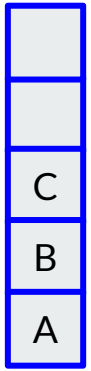


Criação de pilha

```
Pilha* cria_Pilha(){
    Pilha *pi;
    pi = (Pilha*) malloc(sizeof(Pilha));
    if(pi != NULL)
        pi->qtd = 0;
    return pi;
}
```

Inserção Pilha estática

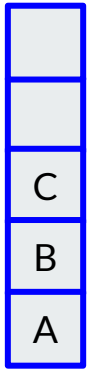
```
// Insere um elemento no topo da pilha (PUSH)
int insere_Pilha(Pilha* pi, struct aluno al){
```



qtd: 3

Inserção Pilha estática

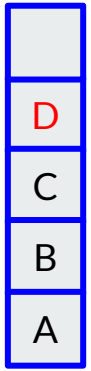
```
// Insere um elemento no topo da pilha (PUSH)
int insere_Pilha(Pilha* pi, struct aluno al){
    if(pi == NULL)
        return 0;
    if(pi->qtd == MAX)//pilha cheia
        return 0;
    pi->dados[pi->qtd] = al;
    pi->qtd++;
    return 1;
}
```



qtd: 3

Inserção Pilha estática

```
// Insere um elemento no topo da pilha (PUSH)
int insere_Pilha(Pilha* pi, struct aluno al){
    if(pi == NULL)
        return 0;
    if(pi->qtd == MAX)//pilha cheia
        return 0;
    pi->dados[pi->qtd] = al;
    pi->qtd++;
    return 1;
}
```



qtd: 4

Consulta topo Pilha estática

```
// Consulta topo (TOP)
int consulta_topo_Pilha( Pilha* pi, struct aluno *al){
   if(pi == NULL || pi->qtd == 0)
       return 0;
   *al = pi->dados[pi->qtd- 1];
   return 1;
}
```

Remoção em Pilha estática

```
// Remove um elemento do topo da pilha (POP)
int remove_Pilha(Pilha* pi){
```

Como ficaria o seu código?

Exercícios:

Usando o exemplo com nome e idade:

1. Inserir 5 pessoas e remover uma da pilha.
2. Imprimir todas as pessoas da pilha.
3. Contar quantas pessoas da pilha tem mais de 20 anos.
4. Buscar uma pessoa pelo nome.
5. Inserir uma pessoa entre a segunda e a terceira posição da pilha.

Operações em pilhas dinâmicas

fonte: https://github.com/arbackes/Estrutura-de-Dados-em-C

Definição dos tipos de dados

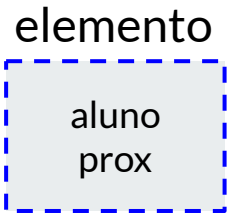
```
// definição de aluno
struct aluno{
    int matricula;
    char nome[30];
    float n1,n2,n3;
};
```



Definição dos tipos de dados

```
// definição de aluno
struct aluno{
   int matricula;
   char nome[30];
   float n1,n2,n3;
};
//Definição do tipo Pilha
typedef struct elemento{
   struct aluno dados;
   struct elemento *prox;
} elemento;
```

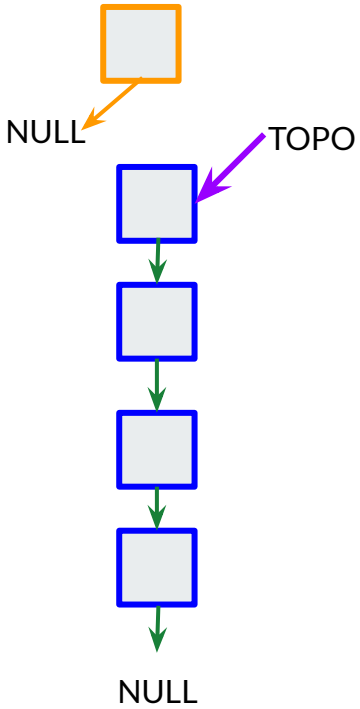




Inserção em pilha dinâmica:

```
int insere_Pilha(elemento **pi, struct aluno al){
   if(pi == NULL)
       return 0;
   Elem* no;
   no = (Elem*) malloc(sizeof(Elem));
   if(no == NULL)
       return 0;
   no->dados = al;


   return 1;
}
```

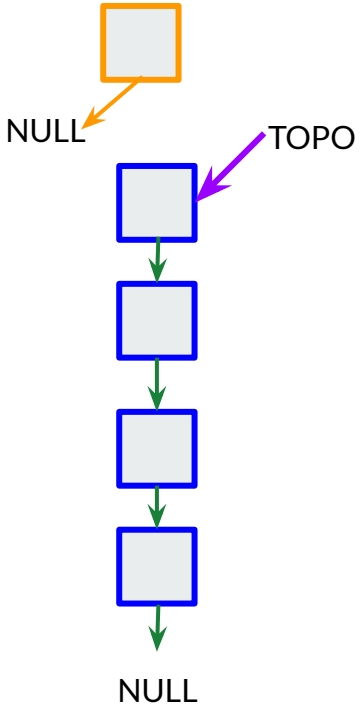


Inserção em pilha dinâmica:

```
int insere_Pilha(elemento **pi, struct aluno al){
    if(pi == NULL)
        return 0;
    Elem* no;
    no = (Elem*) malloc(sizeof(Elem));
    if(no == NULL)
        return 0;
    no->dados = al;

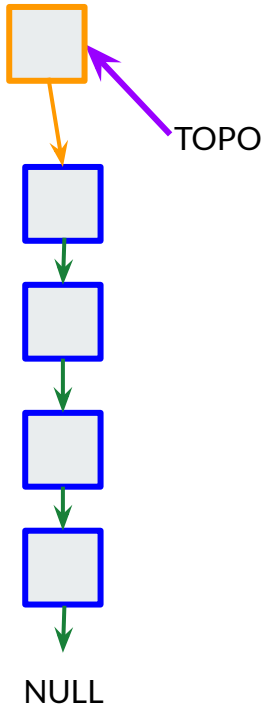

    return 1;
}
```

note o ponteiro duplo para o topo



Inserção em pilha dinâmica:

```
int insere_Pilha(elemento **pi, struct aluno al){
   if(pi == NULL)
       return 0;
   Elem* no;
   no = (Elem*) malloc(sizeof(Elem));
   if(no == NULL)
       return 0;
   no->dados = al;
   no->prox = (*pi);
   *pi = no;
   return 1;
}
```

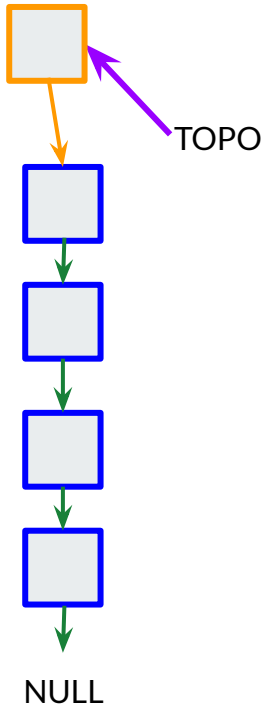


Inserção em pilha dinâmica:

```
int insere_Pilha(elemento **pi, struct aluno al){
   if(pi == NULL)
       return 0;
   Elem* no;
   no = (Elem*) malloc(sizeof(Elem));
   if(no == NULL)
       return 0;
   no->dados = al;
   no->prox = (*pi);
   *pi = no;
   return 1;
}
```

O que acontece se vc inverter estas linhas?





Destruir a pilha:

```
// Remove todos os elementos da pilha:
void libera_Pilha(elemento **pi){
    if (pi != NULL) {
        Elem * no;
        while ((*pi) != NULL) {
            no = *pi;
            *pi = (*pi) -> prox;
            free(no);
        }
    }
}
```

Remover elemento do topo:

```
int remove_Pilha(elemento **pi, struct aluno* al){
   if(pi == NULL)
       return 0;
   if((*pi) == NULL)
       return 0;
   Elem *no = *pi;
   *al = (*pi)->dados
   *pi = no->prox;
   free(no);
   return 1;
}
```

Exercícios - agora com pilha dinâmica:

Usando o exemplo:

1. Inserir 5 pessoas e remover uma da pilha.
2. Imprimir todas as pessoas da pilha.
3. Contar quantas pessoas da pilha tem mais de 20 anos.
4. Buscar uma pessoa pelo nome.
5. Inserir uma pessoa entre a segunda e a terceira posição da pilha.

Exercício: verificação de expressão

Faça um código que leia uma expressão com colchetes, parênteses e chaves como uma string e determine se é válida.

Ex: “{[()]]}” -> inválida

Se ler um caractere de abertura `(`, `[`, `{`, faz um **Push**.

Se ler um de fechamento `)`, `]`, `}`, faz um **Pop** e verifica se o par combina.

Se no fim a pilha estiver vazia, a expressão é válida.